

REMEDIATION AUDIT

Voice Agent v3.0.0

Verification of fixes against the QA pre-test assessment

This audit verifies, finding by finding, what has actually been remediated since the pre-test assessment of 10 September 2026 — which fixes hold up under inspection, which are incomplete, and which have introduced new defects of their own. Every status below was established by reading the code, running the test suite, and starting the application to observe its behaviour.

Prepared for	Voice Agent development team, Technozis
Subject	Voice Agent v3.0.0 — backend, dashboard and deployment manifests
Baseline document	Velu_PreTest_Findings.docx — QA pre-test assessment, 10 September 2026 (22 findings)
Versions examined	v2 (assessment baseline) → v3 (remediation round one) → v4 (remediation round two)
Audit date	15 September 2026
Method	Static review of the full repository diff, plus dynamic verification: test suite executed, application started, authentication / readiness / rate-limit behaviour observed directly
Status	For triage — residual items and regressions to be assigned

1. Executive summary

Of the 22 findings raised in the pre-test assessment, 17 are now fully closed, 4 are partially remediated, and 1 remains untouched. Remediation was delivered in two rounds. The first round addressed the structural problems that blocked testing altogether; the second, and larger, round addressed the defects a caller would actually experience.

Outcome	Count	Meaning
Closed	17	Defect eliminated; remediation verified and sound.
Partially closed	4	Substantive progress, but a material element of the finding remains.
Open	1	No remediation attempted. Lowest severity item in the register.

Principal conclusions

Two of the three release blockers are properly closed. The Twilio entry points (VA-B2) and the unbounded AI calls (VA-C1) have both been remediated well, and in each case the fix was placed where it cannot easily be undone by later work — the signature and token logic in a single shared module, the timeout on the shared client rather than at each call site.

The third release blocker is only half closed, and this is the most significant finding of this audit. The dashboard API is now properly authenticated (VA-B1), but the dashboard website in front of it still has no login and attaches the administrator key to every request it forwards. Both are published to the internet by the same deployment configuration. The exposure has moved one hop rather than been removed.

Remediation introduced five new defects. None is severe, all are inexpensive to correct now, and one of them arises from two fixes interacting — the call-identity fix (VA-C5) silently defeats part of the graceful-shutdown fix (VA-C6). These are set out in section 5.

Twelve fixes were delivered in round two with no accompanying tests. The automated suite is unchanged from round one — 50 tests, byte-for-byte identical files. The test infrastructure exists and passes; only the tests for the new work are missing. This is the audit's primary process observation and is set out in section 6.

Verification performed

The following were observed directly rather than inferred from the code:

- Test suite executed: 50 tests, all passing. The continuous-integration failure identified in the previous review is resolved.
- Unauthenticated request to `/api/dashboard/stats` correctly rejected with 401 Unauthorized.
- Readiness endpoint correctly returned 503 Service Unavailable with `database_connected: false` when the database was unreachable.
- Rate limiter confirmed active — the sixtieth request within one minute returned 429 Too Many Requests.

2. Scope, method and basis of opinion

What was examined

Three snapshots of the repository were compared in full: the version the original assessment was written against, and the two subsequent remediation rounds. Line-ending changes were normalised before comparison, since several files appear entirely rewritten on a naive diff when only their line endings changed.

Coverage included backend application code, database schema and migrations, the dashboard front end, Docker Compose and Kubernetes manifests, dependency manifests, and the continuous-integration workflow.

How status was determined

A finding is recorded as closed only where the remediation addresses the condition described in the assessment, not merely the symptom, and where inspection of the surrounding code gives no reason to doubt it. Where a fix is real but leaves a material element of the finding standing, the status is partial and the remaining element is stated explicitly under residual risk.

Severity is carried forward unchanged from the original assessment. It denotes impact on a production deployment, not difficulty of remediation.

Limitations

- No staging environment was available. Kubernetes runtime behaviour — rolling deployments, autoscaling, ingress and TLS — is assessed from manifest review only.
- No live telephone call was placed. Three remediations warrant confirmation by a real call before release: VA-B2 (the signature check reconstructs the request address from the host header, and a misconfiguration would reject every call silently), VA-C1 (what a caller actually hears when the AI stalls), and VA-C5 (whether a webhook retry now behaves correctly end to end).
- Speech recognition accuracy, latency and load behaviour remain out of scope, as in the original assessment.

3. Remediation status register

All 22 findings, in the order they appear in the assessment. Detailed condition and remediation notes follow in section 4.

ID	Finding	Severity	Status	Remediated
VA-A1	No test infrastructure exists	High	Closed	Round 1
VA-A2	Dependencies constructed at import time	High	Closed	Round 1
VA-A3	Call pipeline is one unreachable 960-line function	High	Partial	Round 1
VA-A4	No database migrations	Medium	Closed	Round 1
VA-A5	Unstructured logs with no call correlation	Medium	Closed	Round 1
VA-B1	Entire dashboard API unauthenticated	Critical	Partial	Round 1

ID	Finding	Severity	Status	Remediated
VA-B2	Twilio webhook signature never validated	Critical	Closed	Round 1
VA-B3	No rate limiting, no concurrency cap	High	Closed	Round 2
VA-B4	Knowledge base wipeable by unauthenticated request	High	Closed	Round 2
VA-B5	Caller data unmasked, no retention policy	High	Partial	Round 2
VA-B6	Prompt injection via uploaded documents	High	Closed	Round 2
VA-C1	No timeout or retry on any OpenAI call	Critical	Closed	Round 2
VA-C2	Cost tracking wrong by construction	High	Partial	Round 2
VA-C3	Deployed dashboard cannot reach the backend	High	Closed	Rounds 1–2
VA-C4	Readiness probe ignores the database	High	Closed	Round 2
VA-C5	No idempotency on Twilio webhook retries	Medium	Closed	Round 2
VA-C6	No graceful shutdown for calls in progress	Medium	Closed	Round 2
VA-C7	Meeting times unvalidated	Medium	Closed	Round 2
VA-D1	Dead code shipping in production	Low	Closed	Round 2
VA-D2	Documentation contradicts the code	Low	Closed	Round 2
VA-D3	Vestigial language parameter	Low	Open	—
VA-D4	Developer-only dependencies in production	Low	Closed	Round 2

4. Findings in detail

Grouped as in the original assessment. Each entry states the condition as it stood at baseline, the remediation delivered, and — where the finding is not fully closed — the element that remains.

A • Testability

Five findings concerning the shape of the code. None describes behaviour a caller would observe; collectively they are the reason no automated testing was possible at baseline.

VA-A1 No test infrastructure exists

Severity: High (blocks testing) **Status:** Closed **Remediated in:** Round 1

CONDITION — WHAT WAS WRONG

The dependency manifest pinned 25 packages required to run the application and none required to test it. There was no test directory, no test framework, no continuous-integration workflow and no test environment configuration.

The only two files named as though they were tests are manual harnesses a developer runs by hand against a microphone. They assert nothing and cannot fail. Automated coverage of the codebase was zero, with no mechanism to change that.

REMEDIATION — WHAT WAS CHANGED

A separate development dependency manifest, a tests directory, a test-runner configuration and a GitHub Actions workflow that executes the suite on every push and pull request.

The suite now holds 50 tests covering the authentication gate, the Twilio signature and stream-token logic, the lazy client construction, the turn-hold state machine and the standalone text helpers. All 50 pass and the CI check is green.

RESIDUAL RISK / AUDITOR NOTE

The suite is narrow. Two testing libraries are pinned but never used (fakeredis, testcontainers), and there are no tests exercising the API through an HTTP client, nor any against a real database or cache. What exists is a scaffold rather than a regression net — which is material given the observation in section 6.

VA-A2 Dependencies are constructed at import time, not injected

Severity: High (blocks testing) **Status:** Closed **Remediated in:** Round 1

CONDITION — WHAT WAS WRONG

Three separate modules each constructed their own OpenAI client at module scope, and the database engine was created the same way. This work executed the moment a file was loaded, before anything had been asked to run.

Consequently the application could not be imported at all without valid API credentials and a reachable database, and a test had no way to substitute a stand-in provider without patching module-level globals. This finding is what made the remainder of the testability work impossible.

REMEDIATION — WHAT WAS CHANGED

A new providers module exposing a single accessor for the OpenAI client, with every former construction site routed through it. The client is built on first genuine use and then reused. The database engine received equivalent treatment.

The refactor was arranged so that all twelve existing database session call sites required no change at all — the session factory was retained as a callable with the same signature. This is a low-disruption change rather than a sweeping one, which is the correct choice here.

VA-A3 The call pipeline is one unreachable 960-line function

Severity: High (blocks testing) **Status:** Partial **Remediated in:** Round 1

CONDITION — WHAT WAS WRONG

Socket handling, audio buffering, voice-activity detection, turn versioning, interruption handling and playback all resided in a single function scope, with the significant state held in closures over local variables.

The logic most likely to carry defects — turn preemption, the hold window that waits for a caller to finish a sentence, and interruption thresholds — could not be exercised at all without establishing a full live call.

REMEDIATION — WHAT WAS CHANGED

The hold-or-answer decision was extracted into its own module as a state class, taking an injectable clock so that elapsed-time behaviour can be tested deterministically and instantly rather than with real delays. Twelve unit tests now cover it.

The extraction is mechanical rather than a rewrite: each method mirrors the closure it replaced, over the same state, with the same arithmetic. That is the appropriate discipline for a refactor made specifically to enable testing.

RESIDUAL RISK / AUDITOR NOTE

Only that one concern was extracted. The principal function remains approximately 910 lines, and interruption handling, turn cancellation and playback are still inaccessible to test.

VA-A4 No database migrations

Severity: Medium (blocks testing) **Status:** Closed **Remediated in:** Round 1

CONDITION — WHAT WAS WRONG

Schema creation was performed by a script that simply created whatever tables the code currently described. There was no versioned schema, no schema version table, and therefore no reproducible test database and no tested upgrade path.

Drift between a developer machine, CI and the managed database was silent. The one schema change made to date shipped as a hand-written SQL file that someone had to remember to execute.

REMEDATION — WHAT WAS CHANGED

Alembic was adopted, with a baseline revision describing the current schema. The documented path is a single upgrade command for a new environment and a stamp command for an existing one, so adoption does not require re-running DDL against live data.

RESIDUAL RISK / AUDITOR NOTE

The baseline was written by hand rather than generated against a live database, and no CI step verifies that it still corresponds to the model definitions. The original create-everything script also remains in place, leaving two sources claiming to define the schema.

VA-A5 Unstructured logs with no call correlation

Severity: Medium (blocks testing) **Status:** Closed **Remediated in:** Round 1

CONDITION — WHAT WAS WRONG

Logging was plain text, and a call identifier appeared only where a developer had manually written it into the message string as a prefix. Any log line emitted by code further from the call path — the reasoning layer, the knowledge search — carried no correlation at all, because that code never receives the identifier.

Diagnosing a failed call therefore meant searching free text, and it was not possible to assert from logs that a call took a particular route or that a fallback fired.

REMEDATION — WHAT WAS CHANGED

A logging utilities module binds the call identifier, and the active turn identifier, to a context variable once at the natural start of a call. A logging filter then stamps both onto every record emitted anywhere during that call, including from modules that never see the identifier as an argument. A JSON formatter emits them as genuine queryable fields.

Approximately sixty existing log statements required no modification, which is the mark of a well-placed fix.

RESIDUAL RISK / AUDITOR NOTE

The web server's own loggers were not reconfigured, so the output stream mixes JSON with plain text. This should be tidied before logs are shipped to an aggregator.

B • Security and privacy

Six findings, including the two most serious in the register. All concern what an unauthorised party could read, change, or spend.

VA-B1 The entire dashboard API is unauthenticated

Severity: Critical **Status:** Partial **Remediated in:** Round 1

CONDITION — WHAT WAS WRONG

The dashboard, settings, documents and analytics routes carried no authentication dependency and no middleware. Every route's sole dependency was a database session.

Any party able to reach the port could read every call transcript, every lead's name, telephone number and email address, and every escalation — and could rewrite the agent's system prompt, which governs what the agent says to customers. The Kubernetes configuration publishes this at a public host name. This is a data-breach-class exposure of personal data belonging to real callers.

REMEDATION — WHAT WAS CHANGED

A dedicated authentication module providing two tiers of shared-secret bearer token. A read key grants read access; a separate administrator key is additionally required for settings writes and knowledge-base changes, on the reasoning that those two govern what the agent tells customers. Keys are compared with a constant-time function rather than ordinary equality, so the secret cannot be recovered by timing.

The design fails closed: an unset key matches nothing, so a deployment that omits configuration locks the dashboard out rather than leaving it open. Verified in this audit — an unauthenticated request returns 401.

RESIDUAL RISK / AUDITOR NOTE

This finding is not closed. The dashboard front end has no login of its own, and to keep functioning it attaches the administrator key to every request it proxies — the implementation comment states this openly. The front end is published to the internet by the same manifest set.

The practical exposure is therefore materially unchanged: an unauthorised party reaches the same data with full administrative rights via the dashboard host instead of the API host. This is the highest-priority item arising from this audit.

VA-B2 Twilio webhook signature is never validated

Severity: Critical **Status:** Closed **Remediated in:** Round 1

CONDITION — WHAT WAS WRONG

Neither telephony entry point was authenticated. The incoming-call webhook never examined the signature header Twilio attaches to every request, and the Twilio library's own validator was installed but never imported. The media-stream socket accepted any connection whatsoever.

A party who learned either address could forge calls. Each forged session spends OpenAI, ElevenLabs and Sarvam credit, writes call and lead records, and can reach the meeting-booking tool to create real calendar entries. There was no cost ceiling and no attribution.

REMEDIATION — WHAT WAS CHANGED

The webhook now validates the signature and rejects failures with 403. Validation fails closed: an absent secret or absent header is always a rejection, never treated as permission to skip the check.

The media stream was the harder half, because Twilio's streaming protocol carries no signature of its own. The remediation follows the assessment's own recommendation to sign the stream address: once a webhook request has been validated, the server mints a short-lived HMAC token bound to that call's identifier and embeds it in the address returned in the TwiML. The socket handler verifies that token before completing the handshake. The token is signed with the existing Twilio secret, so no additional secret enters the estate.

Both halves live in one module so the two sides of the signature cannot drift apart, and so the logic is unit-testable without a Twilio account. Seven tests cover it, including forgery, tampering, expiry and wrong-call-identifier cases.

RESIDUAL RISK / AUDITOR NOTE

Two operational notes. First, the validated address is reconstructed from a configured tunnel host or the inbound host header; if either is stale or rewritten by a proxy, every call is rejected — a total and silent failure that reading the code cannot rule out. A live call must confirm this before release.

Second, the token is not single-use and carries no replay marker, so it remains valid for any number of connections within its 30-second window. This interacts with finding 3 in section 5.

VA-B3 No rate limiting and no concurrency cap

Severity: High **Status:** Closed **Remediated in:** Round 2

CONDITION — WHAT WAS WRONG

Nothing bounded concurrent calls, requests per client, or document uploads. The database connection pool permitted roughly fifteen connections, so approximately fifteen simultaneous calls exhausted it and every subsequent turn began failing on database access — and every conversational turn requires the database.

In combination with the forged-call exposure above, this was also an inexpensive route to an unbounded provider bill.

REMEDIATION — WHAT WAS CHANGED

Two distinct controls for two distinct problems. Live calls now pass a semaphore admitting a configurable maximum (default 20) and refusing the remainder. The refusal occurs before any call record is written or any paid provider is contacted, so a rejected call incurs no cost — the correct ordering.

The database pool was resized to the same cap with proportional overflow, so the two numbers now agree rather than contradicting one another. Separately, the HTTP surface received a per-client rate limit with a stricter limit on the incoming-call webhook. Verified active in this audit.

RESIDUAL RISK / AUDITOR NOTE

The limiter identifies clients by the immediate network address and does not consult the forwarded-for header. See findings 1 and 2 in section 5 — this is the weakest element of an otherwise sound remediation.

VA-B4 Knowledge base can be wiped by an unauthenticated request

Severity: High **Status:** Closed **Remediated in:** Round 2

CONDITION — WHAT WAS WRONG

A single unauthenticated request permanently deleted every chunk belonging to a named source, and the name was taken directly from the request path with no validation. The upload counterpart was equally open, permitting anyone to add content the agent would subsequently present to callers as company fact.

REMEDIATION — WHAT WAS CHANGED

Three layers. Upload and delete now require the administrator tier introduced under VA-B1. Deletion additionally requires an explicit confirmation parameter, so the operation cannot be triggered inadvertently. The source name is now validated against the sources that actually exist rather than trusted.

RESIDUAL RISK / AUDITOR NOTE

Deletion remains immediate and permanent rather than a soft delete, and the existence check performs a full scan of the document table on each call. Neither is urgent.

VA-B5 Caller personal data stored and logged unmasked, with no retention policy

Severity: High **Status:** Partial **Remediated in:** Round 2

CONDITION — WHAT WAS WRONG

Masking was applied only at the dashboard API boundary, on read. Beneath it the database held full telephone numbers, email addresses and complete transcripts indefinitely, and the log stream emitted full telephone numbers at information level from four separate modules.

There was no retention window, no deletion endpoint and no subject-access path. The company is UK-based, placing this within UK GDPR scope.

REMEDIATION — WHAT WAS CHANGED

A privacy utility providing telephone and email masking, now applied at every point a number was being logged. The reasoning is sound and worth recording: logs typically flow to systems with looser access controls than the database, making the log stream the higher-risk copy to address first.

A right-to-erasure function deleting every call, transcript, lead, meeting, escalation and sentiment record for one telephone number, exposed as an administrator-only endpoint behind an explicit confirmation. Deletion order was worked out correctly — no foreign key cascades, so children must be removed before parents.

A retention purge script deleting calls older than a configured window, with a dry-run mode.

RESIDUAL RISK / AUDITOR NOTE

The finding is not closed. Nothing schedules the purge script — no cron job exists in any manifest — so the retention setting currently has no effect whatsoever. The default window is explicitly labelled a placeholder rather than a decision, and the assessment's point that this requires legal rather than purely technical input still stands.

The database continues to store identifiers in clear text by design, and the erasure function matches the telephone number as an exact string, so a number persisted in a different format will not be found.

VA-B6 Prompt injection through uploaded knowledge-base documents

Severity: High **Status:** Closed **Remediated in:** Round 2

CONDITION — WHAT WAS WRONG

Retrieved document passages were inserted into the agent's system prompt under a plain heading, with nothing distinguishing them from the prompt's own instructions. To the model, a sentence retrieved from an uploaded file was indistinguishable from a rule written by the business.

Instructions concealed in an uploaded document therefore became instructions the agent followed on live calls. The existing guardrails offered no protection, since they only ever inspect what the caller said.

REMEDICATION — WHAT WAS CHANGED

Retrieved content is now enclosed in explicit begin and end markers, preceded by an unambiguous instruction that everything between them originated from an uploaded document and must be treated as data to cite, never as instructions to follow — with a specific direction to disregard anything resembling a command, a request to ignore the rules, or a change of role.

Combined with the upload restriction delivered under VA-B4, this closes both the route in and the effect. The output guardrail continues to run over whatever is produced.

C · Correctness and operations

Seven findings describing incorrect behaviour in normal operation — the group whose effects a caller or the business would directly experience.

VA-C1 No timeout or retry on any OpenAI call

Severity: Critical **Status:** Closed **Remediated in:** Round 2

CONDITION — WHAT WAS WRONG

Every other external dependency had a bounded wait. Speech recognition had explicit idle timeouts; the speech path had a turn timeout. The language-model path — which runs on the critical path of every single conversational turn — had none.

A slow or hanging response therefore left a live caller listening to silence, with no fallback message and no escalation, until the telephony provider eventually tore the call down. The router, the specialist agents and the embedding call were all exposed.

A second defect sat beneath it. The failure path did produce a spoken apology undertaking that someone would call back, but no escalation record was ever created. The undertaking was one the system had no mechanism to keep.

REMEDICATION — WHAT WAS CHANGED

An explicit per-request timeout of 8 seconds and one bounded retry, configured once on the shared client introduced under VA-A2 rather than at each call site. This placement is the right one: every existing and future call through that client is covered automatically, and a later addition cannot silently reintroduce an unbounded wait.

The failure path now creates the escalation record it promises, wrapped so that a failure to record the escalation cannot itself break the call.

RESIDUAL RISK / AUDITOR NOTE

The bound applies to each request, not to the caller's total wait. A single turn issues several requests in sequence — routing, the specialist, a tool follow-up, a retry — each attracting its own 8 seconds plus retry. The aggregate a caller

could experience before the fallback is spoken substantially exceeds the intent, against a turn-preemption window of 3 seconds. See finding 4 in section 5.

VA-C2 Cost tracking is wrong by construction

Severity: High **Status:** Partial Remediated in: Round 2

CONDITION — WHAT WAS WRONG

The speech cost accumulator was initialised to zero and never incremented anywhere in the codebase; at the end of every call it faithfully wrote that zero to the record. Speech-to-text usage, which runs continuously throughout every call, was not metered at all.

Every per-call and dashboard cost figure therefore reflected language-model tokens only. Speech is likely the larger share of the bill, so the figure presented to the business was not merely imprecise but systematically understated, and unusable for pricing or budgeting.

REMEDIATION — WHAT WAS CHANGED

The streaming speech session now counts the characters it submits and reports its own cost at the rate for whichever provider actually handled the turn. That total is collected in a block that executes however the turn ended — normal completion, interruption, timeout or failure — which is the correct placement.

Speech recognition, which is not attributable to any single turn, is metered against call duration at the point the call record is finalised.

RESIDUAL RISK / AUDITOR NOTE

All three unit rates default to zero, because the codebase cannot know the negotiated commercial rates. Until they are populated from the provider billing portals, the dashboard presents the same incorrect figure as before, for a different reason. The team documented this candidly in the code, but it is a live trap for whoever deploys next.

The non-streaming speech path remains unmetered, and the database column and API field retain a provider-specific name while now representing all speech cost.

VA-C3 The deployed dashboard cannot reach the backend

Severity: High **Status:** Closed Remediated in: Rounds 1–2

CONDITION — WHAT WAS WRONG

All browser traffic is proxied server-side by the dashboard. The proxy destination fell back to the local machine on port 8000, which is correct when both processes run on one host but wrong in every containerised deployment, where the dashboard and backend are separate containers. Every dashboard page would fail to load data.

Meanwhile the variable that was correctly configured in three places was read by no source file in the project — inert configuration presenting as functional.

REMEDIATION — WHAT WAS CHANGED

The configuration-level rewrite was replaced with a server-side route handler reading the correct internal address. The change was necessary in any event: a rewrite cannot attach a header, so it could not have carried the credential the backend now requires under VA-B1. One change resolved two obstacles.

In round two the inert variable was removed from all three locations, as the assessment recommended.

VA-C4 Readiness probe ignores the database

Severity: High **Status:** Closed Remediated in: Round 2

CONDITION — WHAT WAS WRONG

The readiness endpoint reported on Redis — a performance layer the system is explicitly designed to survive the loss of — and said nothing about Postgres, without which it cannot serve a single turn. A backend with an unreachable

database therefore passed readiness and continued receiving calls. The probe checked the optional dependency and skipped the mandatory one.

REMEDIATION — WHAT WAS CHANGED

A minimal connectivity query was added, returning 503 on failure so the orchestrator withdraws the instance from service. Redis continues to be reported but cannot fail the probe — precisely the split the assessment recommended. Verified in this audit.

VA-C5 No idempotency on Twilio webhook retries

Severity: Medium **Status:** Closed **Remediated in:** Round 2

CONDITION — WHAT WAS WRONG

The telephony provider re-posts the incoming-call webhook on timeout, which is normal and expected behaviour. Each attempt caused the system to mint a fresh random identifier bearing no relationship to the provider's own identifier for that call.

One real conversation could therefore produce two call records and two partial transcripts, overstating call counts and cost totals and splitting a single conversation across two entries.

REMEDIATION — WHAT WAS CHANGED

The provider's own call identifier is now used as the system's call identifier. Because the stream token from VA-B2 is signed against that identifier, it is verified before use and cannot simply be asserted by a caller. Every table is now naturally keyed to the one real call.

As a secondary measure, call creation and transcript storage were both made repeat-safe: an existing record is returned or updated rather than a second being inserted. This is necessary as well as prudent — with the identifier now fixed, a plain insert on a genuine retry would fail on the primary key.

RESIDUAL RISK / AUDITOR NOTE

This remediation has an unintended interaction with VA-C6. See finding 3 in section 5.

VA-C6 No graceful shutdown for calls in progress

Severity: Medium **Status:** Closed **Remediated in:** Round 2

CONDITION — WHAT WAS WRONG

Shutdown closed the Redis and pooled HTTP clients only. Active sessions were never tracked, so a rolling deployment or a scale-in event terminated live callers mid-sentence.

The loss was greater than the interruption itself: transcript and cost are written only in the stream handler's final block, so a truncated call left no record that it had occurred. With two replicas and autoscaling to ten, this would have occurred in routine operation rather than exceptionally.

REMEDIATION — WHAT WAS CHANGED

Three coordinated parts. Active calls are now tracked. On shutdown the application first refuses new stream connections, then waits a bounded period for in-flight calls to complete naturally — so their finalisation block does run — before tearing down the clients those calls depend upon.

A matching termination grace period was added to the deployment manifest, deliberately set above the application's own drain budget. Without it the platform default would have raced the drain and terminated the process mid-way, reproducing the original defect. Identifying that interaction is the difference between a remediation and one that actually holds.

RESIDUAL RISK / AUDITOR NOTE

The tracking structure is defeated by the VA-C5 remediation. See finding 3 in section 5.

VA-C7 Meeting times are unvalidated naive datetimes

Severity: Medium **Status:** Closed **Remediated in:** Round 2

CONDITION — WHAT WAS WRONG

Whatever date and time the model produced was accepted without check. Nothing rejected a booking in the past, outside business hours, or years ahead. The sole defence was the model reading the current date correctly from its prompt — which had already failed once in production, placing a booking in 2023.

A further defect compounded it: where the value failed to parse at all, the code silently substituted the current time and booked that. A complete parsing failure produced a real calendar entry rather than an error.

REMEDIATION — WHAT WAS CHANGED

A validation function checking four conditions before anything is written: that the value parses, that it is in the future, that it falls within a configurable booking horizon, and that it falls within configurable business hours. Comparison is performed in the business's own timezone, which is correctly identified as the only timezone the codebase actually knows.

The silent substitution is removed. Rejections return a specific, actionable reason to the model — including the current time and an explicit instruction not to select a date itself — so the agent recovers by asking the caller again rather than failing the call.

D • Hygiene

Four low-severity findings. Low severity denotes low production impact, not low value — three of these consumed QA time on every investigation.

VA-D1 Dead code shipping in production

Severity: Low **Status:** Closed **Remediated in:** Round 2

CONDITION — WHAT WAS WRONG

Four elements that could not execute: a webhook for an alternative telephony provider whose router was never mounted, an empty vestigial package, an audio conversion routine unreachable under the default configuration, and a knowledge-search tool defined but bound to no agent's tool list.

The cost is real even where the risk is not: anyone reading the code to understand behaviour is misled, and triage time is spent on paths that cannot run.

REMEDIATION — WHAT WAS CHANGED

The unused webhook module, the empty package and the orphaned search tool were removed, with a comment retained at each removal site recording why the code was unreachable. That note is worth more than a silent deletion — it prevents the same code being reintroduced.

VA-D2 Documentation contradicts the code

Severity: Low **Status:** Closed **Remediated in:** Round 2

CONDITION — WHAT WAS WRONG

The README described a single-call architecture that no longer exists and listed five of the eight dashboard pages. The setup guide contained a developer's personal Windows path and two real customer telephone numbers.

The material item was caller-facing: the deployed greeting offered Tamil, Hindi and English. This is an English-only build. Every caller was being promised two languages the agent cannot speak.

REMEDIATION — WHAT WAS CHANGED

The deployed greeting was corrected to match the build and no longer offers languages the agent does not support. The README was reconciled with the orchestrator-only design, and the guide had the personal path and the real telephone numbers removed.

VA-D3 Vestigial language parameter threaded through the whole stack

Severity: Low **Status:** Open **Remediated in:** —

CONDITION — WHAT WAS WRONG

Language is hard-coded to English at the point of capture, yet a language value continues to be passed from the audio layer through the agent and orchestrator to speech output — carried through the entire stack as a constant that cannot hold any other value.

It is harmless in operation. The risk is that a future contributor infers from its presence that multilingual support functions, and builds on that assumption.

REMEDIATION — WHAT WAS CHANGED

No remediation attempted. This is the only finding of the 22 with no work against it, which is defensible given it is the lowest-impact item in the register.

RESIDUAL RISK / AUDITOR NOTE

Recommended disposition: remove the parameter, or re-establish it as a genuine setting with tests behind it. Either is acceptable; leaving it as an inert constant is the option that accrues risk.

VA-D4 Developer-only dependencies pinned in production requirements

Severity: Low **Status:** Closed **Remediated in:** Round 2

CONDITION — WHAT WAS WRONG

Two packages supporting microphone capture were declared as production dependencies. They are used by exactly one manual script run on a developer's own machine. Every production image therefore carried native audio libraries and their system dependencies for no operational purpose.

REMEDIATION — WHAT WAS CHANGED

Both were moved to the development dependency manifest created under VA-A1. This also resolved the continuous-integration failure identified in the previous review, which arose from precisely this pairing.

5. Defects introduced by the remediation

Remediation carries its own risk, and this section records it. One defect introduced in round one has since been resolved; five introduced in round two remain open. None is severe. All are inexpensive to correct now and become progressively more expensive once the system is in service.

#	Defect	Origin	Effect
R0	CI collected a manual harness as a test	VA-A1	Resolved in round two. Every push reported a failure despite all real tests passing.
R1	Rate limit keyed on the proxy address	VA-B3	All dashboard users share a single allowance; the webhook limit becomes a global cap on inbound calls.
R2	Health endpoints are rate-limited	VA-B3	A throttled liveness probe reads as a dead application and triggers a restart loop.
R3	Shutdown tracker loses count on duplicates	VA-C5 + VA-C6	Drain can conclude while a call is still active, reproducing the defect VA-C6 was raised to fix.
R4	Timeout bounds the request, not the turn	VA-C1	Aggregate caller wait before the fallback substantially exceeds the intended bound.
R5	Connection maths ignores replica count	VA-B3	At maximum autoscale the estate demands roughly 400 database connections.

R0 — Continuous integration failed on its first run (resolved)

The test-runner configuration did not restrict where tests were collected from, so the runner picked up the manual microphone harness as though it were a test. That script requires audio hardware the CI runner does not provide, and errored. The result was a failing check on every change while all 50 genuine tests passed.

Corrected in round two by a single configuration line restricting collection to the tests directory. Verified in this audit: 50 passed, check green.

R1 — The rate limit identifies the wrong client

Client identity is taken from the immediate network address, with no consultation of the forwarded-for header. Every dashboard request reaches the backend from the dashboard server rather than from the person operating it, so all users share one allowance between them and a small number of concurrent operators can exhaust it for everybody.

The same applies to the incoming-call webhook: its limit observes the gateway address, converting a per-client control into a global ceiling on inbound call volume. Both are a configuration change to the key function rather than a redesign.

R2 — The readiness and liveness endpoints are themselves rate-limited

The default limit applies to both probe endpoints. At the configured probe intervals this sits comfortably within the allowance, so it is not currently failing. The exposure is that any additional polling from the same source address pushes it over, and a throttled liveness probe is indistinguishable to the orchestrator from a dead application — producing a restart loop whose cause would be materially difficult to diagnose. Both endpoints should be exempted.

R3 — The shutdown tracker can lose count, arising from two fixes interacting

Active calls are recorded in a set keyed on the call identifier. The premise of VA-C5 is that a webhook retry can legitimately produce two concurrent stream connections bearing the same provider call identifier — and that identifier is now the key.

When the first of the two connections completes, it removes the key while the second remains active. The drain logic then observes an empty set, concludes that all calls have finished, and permits the process to exit — terminating precisely the caller the drain was introduced to protect. A reference count in place of a set resolves it.

A related observation: nothing rejects the duplicate connection itself. Both would greet the caller, and because transcript storage now overwrites, the surviving transcript is whichever connection completes last. A guard admitting only one live connection per call identifier is the fuller fix, and also closes the token replay window noted under VA-B2.

R4 — The language-model timeout bounds the request, not the caller's wait

Eight seconds per request is a reasonable figure. A single conversational turn, however, issues several requests in sequence — routing, the specialist agent, a tool follow-up, and a retry where applicable — each attracting its own timeout and retry allowance.

Aggregated, the interval a caller may wait before hearing the fallback substantially exceeds what the remediation intends, against a turn-preemption window of three seconds. The assessment asked for a budget with a spoken fallback when it is exhausted; what exists is a per-request limit with no budget above it.

R5 — Connection sizing does not account for replica count

Each backend instance now holds up to forty database connections, correctly proportioned to the twenty-call concurrency cap. The deployment scales automatically to ten instances, giving roughly four hundred connections in aggregate — considerably beyond what the managed database's connection pooler will accept.

The per-instance figure is right; nothing accounts for the number of instances. Relatedly, the rate limiter holds its counters in process memory, so the effective allowance is multiplied by the replica count and resets on every restart. Redis is already provisioned and would address both.

6. Process observation — test coverage of the remediation

Twelve findings were remediated in round two. The automated test suite is unchanged: 50 tests, and the test files are byte-for-byte identical to round one. No test accompanies any of the new work.

This is recorded as the audit's principal process observation rather than as a defect, because nothing is presently broken by it. Its significance is prospective. The code changed in round two is disproportionately the kind that tests protect cheaply and effectively:

- The meeting-time validator is a self-contained function taking a date and a time and returning an acceptance or a reason for refusal. Its four conditions — parses, future, within horizon, within business hours — are four short tests.
- The repeat-safety helpers introduced under VA-C5 are thin database functions with one branch each.
- The telephone and email masking functions are pure and have obvious edge cases — short values, absent values, malformed addresses.
- The interaction between the authentication tiers and the rate limiter is exactly the kind of cross-cutting behaviour that regresses silently.

The infrastructure to run these tests exists, is configured, and reports green. Only the tests themselves are absent. The estimated effort for the items above is in the order of an hour, and it protects twelve remediations against silent regression when a neighbouring line is edited months from now.

A secondary observation: none of the five regressions in section 5 would have been caught by a test, since all are configuration or concurrency behaviour rather than logic. They were found by inspection. That argues for retaining inspection alongside the suite rather than for expanding the suite alone.

7. Recommendations

In priority order. The first three are recommended before any deployment reachable from the internet.

#	Recommendation	Relates to	Priority
1	Place a login in front of the dashboard, or issue per-user credentials through the proxy in place of the shared administrator key.	VA-B1	Before release
2	Place one live telephone call to confirm the signature validation reconstructs the request address correctly. The failure mode is total and silent.	VA-B2	Before release
3	Populate the three speech cost rates from the provider billing portals, or relabel the dashboard field so it does not present as total cost.	VA-C2	Before release
4	Write tests for the round-two remediations, beginning with the meeting-time validator, the repeat-safety helpers and the masking functions.	Section 6	High
5	Correct the rate limiter: read the forwarded client address, exempt both probe endpoints, and move counters to Redis.	R1, R2, R5	High
6	Replace the shutdown tracking set with a reference count, and reject a second concurrent connection for a call identifier already in progress.	R3	High
7	Introduce a whole-turn time budget with the spoken fallback triggered on exhaustion, rather than relying on per-request limits aggregating acceptably.	R4	Medium
8	Size the database pool against the maximum replica count rather than a single instance.	R5	Medium

#	Recommendation	Relates to	Priority
9	Schedule the retention purge script, and confirm the retention period with legal input.	VA-B5	Medium
10	Add a CI step verifying the Alembic baseline still corresponds to the model definitions, and retire the parallel create-everything script.	VA-A4	Medium
11	Continue the extraction of the call pipeline — turn preemption and interruption handling next.	VA-A3	Low
12	Remove the vestigial language parameter, or re-establish it as a genuine setting with tests.	VA-D3	Low

8. Overall assessment

The remediation is substantially sound. Seventeen of twenty-two findings are properly closed in two rounds, and the quality of the individual fixes is consistently better than the minimum the findings required. Several show real engineering judgement rather than compliance: placing the timeout on the shared client so it cannot be bypassed by later work; recognising that the termination grace period had to exceed the application's own drain budget; making the authentication fail closed; keeping both halves of the signature logic in one module so they cannot diverge.

Three qualifications temper that. The most serious finding in the register, VA-B1, is closed on the server and left open at the front door, and the deployment configuration publishes both — so the exposure has been relocated rather than removed. Twelve remediations shipped without tests, in a codebase whose testing was itself one of the original findings. And the round-two work introduced five defects of its own, one arising from two fixes interacting in a way neither author would have seen alone.

None of the outstanding items is difficult. All are well understood, and the majority are single-file changes. The recommended sequence is section 7: close the front door, confirm the signature path with a live call, then write the tests that will hold this work in place.

Prepared as a follow-up to the QA pre-test assessment of 10 September 2026. Severity throughout is carried forward from that document and denotes impact on a production deployment, not difficulty of remediation. Status reflects the v4 snapshot as examined on 15 September 2026. **9. Addendum — independent full-codebase sweep (16 September 2026)**

This addendum is a separate audit event, appended to the report rather than merged into it. It does not revise the conclusions of sections 1–8, which stand as the record of the 10 September assessment and its two remediation rounds. What follows was commissioned independently of that register: a from-scratch review of the entire repository — backend, dashboard, Docker and Kubernetes manifests — for security, performance and completeness defects, without reference to which of the original 22 findings had or had not been raised. Sixteen were closed the same day, by direct code and configuration change, verified by the full test suite, a fresh production build of the dashboard, and (for the one item touching schema history) the Alembic revision chain. Two matters were put to the client rather than fixed unilaterally, because closing them meant a genuine architectural decision, not a bug fix; both are recorded in section 9.5.

9.1 Scope and method

Coverage: `backend/` in full (API routers, the LangGraph orchestrator, the voice pipeline, the database layer, the guardrails module, dependency manifests), `frontend/` in full (the Next.js dashboard and its server-side proxy to the backend), `docker-compose.yml`, every manifest under `k8s/`, and both `.env.example` templates. Each finding below was established by reading the implicated module directly, not by pattern-matching against a checklist; each remediation was verified either by an automated test run, a production build, or direct inspection of the changed file's behaviour (e.g. parsing every edited YAML manifest, walking the new Alembic revision chain).

Sixteen findings were remediated the same session. Two were referred to the client for a decision rather than closed outright:

- Whether the Kubernetes ingress rule that publishes the backend's Twilio entry points should be removed entirely or narrowed — narrowing was chosen (section 9.4, E1), since Twilio must reach `/incoming-call` and `/stream` directly and removing the rule outright would have broken inbound calls.
- Whether to build a login for the dashboard now, closing VA-B1 from the original register outright — the client elected not to, on this occasion (section 9.5).

9.2 Summary

Outcome	Count	Meaning
Fixed this session	16	Remediated and verified as described in 9.4.
Referred to client, resolved by decision	2	Not a defect to patch silently — the ingress scope and the dashboard login both required the client's own judgement; see 9.5.

Verified directly: full backend test suite, 50 passed, no regressions; the dashboard's production build (`next build`) compiles cleanly with all eleven routes present; the new Alembic revision (0001 → 0002) resolves to a single head with no branch.

9.3 Register

ID	Finding	Severity	Status
E1	Kubernetes ingress published the entire backend, not just the Twilio entry points	High	Closed
E2	Dashboard proxy had no CSRF or path-traversal defence	Medium	Closed
E3	Containers ran as root; no Kubernetes <code>securityContext</code> anywhere	Medium	Closed
E4	Redis was unauthenticated, and published to the host in Docker Compose	Medium	Closed
E5	<code>CORS_ALLOWED_ORIGINS</code> was absent from the Kubernetes ConfigMap	Medium/High	Closed
E6	<code>python-multipart</code> was pinned to a version with a known denial-of-service CVE	Low	Closed
F1	Output-guardrail leak detection covered two literal phrasings only	Medium	Closed
F2	End-of-speech detection rescanned the whole call buffer on every audio frame	High	Closed

ID	Finding	Severity	Status
F3	PDF text extraction blocked the event loop shared with live calls	Medium	Closed
F4	Two knowledge-base queries loaded every stored embedding into Python to answer a count	Medium	Closed
F5	brain/tools.py used inline <code>__import__</code> in place of ordinary imports	Low	Closed
G1	The Alembic baseline had drifted from database/models.py	High	Closed
G2	SUPABASE_URL / SUPABASE_KEY were configuration fields nothing read	Medium	Closed
G3	Twenty-three environment variables config.py reads were undocumented	Low	Closed
H1	No ESLint configuration existed; <code>next lint</code> was a no-op	Medium	Closed
H2	A Settings change did not propagate to the rest of the dashboard without a reload	Medium	Closed
H3	Four dashboard views were exposed to out-of-order response races	Medium	Closed
H4	The analytics page's "live" feed never actually refreshed	Low	Closed
H5	A settings label was not bound to its control; a failed escalation resolve gave no visible error	Low	Closed
H6	Six installed dependencies were never imported anywhere in the dashboard	Low	Closed

9.4 Findings in detail

E · Deployment and network exposure

E1 Kubernetes ingress published the entire backend, not just the Twilio entry points

Severity: High **Status:** Closed

Condition. The `api.yourdomain.com` ingress rule matched path: `/` with prefix matching, which put every backend route on the public internet: the dashboard/admin API under `/api/*` (gated only by a shared bearer key), and the

unauthenticated `/docs`, `/openapi.json`, `/health` and `/ready` endpoints. Twilio does genuinely need to reach two unprefixed routes directly — `/incoming-call` and `/stream` — so the rule could not simply be deleted without breaking inbound calls.

Remediation. The rule was narrowed to exact-path matches on precisely those two routes. Anything else arriving at that host now receives no matching rule (a 404) rather than reaching the backend. A comment in the manifest records why the two exceptions exist.

E2 Dashboard proxy had no CSRF or path-traversal defence

Severity: Medium **Status:** Closed

Condition. The Next.js route handler that proxies every dashboard request to the backend (attaching the administrator credential itself, so the browser never holds it) performed no origin check. A form on an external page could POST to, for instance, `/api/settings` and have the browser carry the request there with the credential attached server-side. Separately, a path segment of `..` (including its percent-encoded form) was not rejected before being joined into the backend URL, which could walk a request out of the `/api/` prefix the proxy is meant to enforce and reach an unauthenticated root-level route.

Remediation. Non-GET requests are now rejected with 403 unless the request's own `Origin` (or, failing that, `Referer`) header names the same host as the request itself arrived on — derived from the request, not a hardcoded domain, so the check holds across local development, Docker Compose and the Kubernetes ingress unmodified. Any path segment equal to `.` or `..` is rejected with 400 before the backend URL is constructed, as defence in depth alongside the E1 narrowing.

E3 Containers ran as root; no Kubernetes securityContext anywhere

Severity: Medium **Status:** Closed

Condition. Neither the backend nor the frontend Dockerfile declared a `USER`, so both images ran as root by default, and none of the three Kubernetes workloads (backend, frontend, Redis) set a pod- or container-level `securityContext`.

Remediation. Both Dockerfiles now create an unprivileged user and switch to it before the final command. Both application Deployments carry `runAsNonRoot`, a fixed non-root UID matching the image, `allowPrivilegeEscalation: false` and a full capability drop. The Redis StatefulSet required a separate judgement call: its manifest overrides the official image's command, which bypasses that image's own root-to-redis-user step-down, so the manifest now pins the image's known UID (999) explicitly, including `fsGroup` so the persistent volume remains writable under that UID.

E4 Redis was unauthenticated, and published to the host in Docker Compose

Severity: Medium **Status:** Closed

Condition. Redis held session state, the LLM response cache, the RAG cache and the event bus with no password of any kind, and `docker-compose.yml` published its port to the host network in addition to the internal Compose network that is all the backend actually requires.

Remediation. Both the Compose file and the Kubernetes manifests now require a password: Compose fails to start at all without `REDIS_PASSWORD` set (a deliberate hard failure, not a silent fallback), and the host port mapping was removed — only the backend container can reach Redis, over the internal network. The Kubernetes side takes the same password from the same secret referenced by the backend, passed to `redis-server` via Kubernetes' own command-substitution mechanism, with the readiness probe updated to authenticate when it checks liveness.

E5 CORS_ALLOWED_ORIGINS was absent from the Kubernetes ConfigMap

Severity: Medium, in practice High on first production deploy **Status:** Closed

Condition. The application's own default for this setting is `localhost`, intended for local development. Nothing in the Kubernetes ConfigMap overrode it, so a cluster deployment would silently keep that default — and the production dashboard's credentialed requests, arriving from the real ingress host, would be rejected by the backend's own CORS policy. This is the kind of defect that is invisible until the first real deployment and then affects every request.

Remediation. The ConfigMap now sets `CORS_ALLOWED_ORIGINS` to the same host used elsewhere in the manifests for the dashboard, with a comment instructing the deployer to replace the placeholder domain.

E6 python-multipart was pinned to a version with a known denial-of-service CVE

Severity: Low **Status:** Closed

Condition. `requirements.txt` pinned `python-multipart==0.0.12`, which corresponds to CVE-2024-53981 — a malformed-input denial-of-service in multipart form parsing, reachable through the document-upload endpoint.

Remediation. Bumped to a version past the fix.

F · Runtime correctness and performance

F1 Output-guardrail leak detection covered two literal phrasings only

Severity: Medium **Status:** Closed

Condition. The pattern list intended to catch the model reciting its own system prompt or a caller’s payment details back verbatim held exactly two fixed phrasings. Any rephrasing of either — “here is what I was told to do” in place of “my system prompt is” — passed through unblocked, was spoken to the caller and written to storage.

Remediation. Expanded to seventeen patterns spanning the common phrasings of a system-prompt disclosure and of card, CVV and Social Security Number leakage, wired into the existing storage-side masking alongside the card-number masking that was already present.

F2 End-of-speech detection rescanned the whole call buffer on every audio frame

Severity: High **Status:** Closed

Condition. The function deciding whether a caller had finished speaking carried a docstring claiming constant-time, tail-only behaviour, but in fact called the sustained-speech check against the *entire* accumulated buffer on every roughly 20 millisecond frame of audio. Because that buffer grows for the length of an utterance (bounded at 20 seconds), the actual cost was quadratic in call length — synchronous CPU work, on the same event loop handling every other concurrent call, growing worse for every frame of a long utterance.

Remediation. Bounded the scan to a fixed trailing window — the silence-detection window plus a three-second margin, sized to comfortably exceed the sustained-speech check’s own 800-millisecond requirement — so the cost per frame is now constant regardless of how long the utterance has run. Verified against synthetic audio that ordinary speech-then-pause still triggers at the same instant as before, and that a deliberately adversarial case (intermittent noise blips spread over several seconds, repeatedly resetting the old code’s silence counter) now falls back to the existing 20-second hard cutoff rather than hanging indefinitely — a narrower, explicitly accepted trade-off in exchange for removing the quadratic cost.

F3 PDF text extraction blocked the event loop shared with live calls

Severity: Medium **Status:** Closed

Condition. The document-upload endpoint extracted PDF text synchronously, inline in an `async` request handler, on the single event loop that also carries every live phone call’s audio. A sufficiently large upload (up to the existing size limit) could stall that loop for the duration of extraction.

Remediation. The extraction call now runs in a worker thread, off the event loop, with its return value and error behaviour unchanged.

F4 Two knowledge-base queries loaded every stored embedding into Python to answer a count

Severity: Medium **Status:** Closed

Condition. The functions listing and deleting knowledge-base document sources both selected entire rows — including the stored vector embedding for every chunk — purely to group or filter by a metadata field, then discarded the embeddings in Python. The cost scales with the size of the knowledge base and transfers data the query never needed.

Remediation. Both now perform the grouping and filtering in SQL, so no embedding is ever loaded into the application for these operations. The externally visible shape of both functions' results is unchanged.

F5 brain/tools.py used inline `__import__` in place of ordinary imports

Severity: Low **Status:** Closed

Condition. The escalation tool constructed two of its dependencies via `__import__` calls at the point of use, rather than ordinary module imports — a pattern that usually signals a circular-import workaround.

Remediation. Checked for an actual circular dependency and found none; converted both to ordinary top-level imports and confirmed the module still imports cleanly.

G · Data model and configuration hygiene

G1 The Alembic baseline had drifted from database/models.py

Severity: High **Status:** Closed

Condition. The single baseline migration creates `calls.language_detected` and `leads.language`, columns that do not exist on the corresponding model classes. A by-hand SQL script elsewhere in the repository dropped them on at least one occasion, but that script sits outside Alembic's own version history — so a fresh environment running `alembic upgrade head` still creates two columns the application never reads or writes, with no migration recording their removal.

Remediation. Rather than editing the baseline — unsafe, since it may already be applied against a live database — a new revision was added that drops both columns, with a downgrade path that restores them exactly as the baseline defined them. The existing by-hand SQL script was annotated as superseded by the new migration. The resulting chain was walked with Alembic's own tooling and resolves to one head.

G2 SUPABASE_URL / SUPABASE_KEY were configuration fields nothing read

Severity: Medium **Status:** Closed

Condition. Both were declared as settings, listed in the environment template, and named in the setup instructions as required values — but no code anywhere read either. The actual database connection is made entirely through `DATABASE_URL`. A deployer following the documentation as written could reasonably populate the two dead fields, skip `DATABASE_URL`, and have nothing work.

Remediation. Removed both fields and their template entries; the setup documentation now states plainly that `DATABASE_URL` alone is required, Supabase being one way to obtain a Postgres connection string among others.

G3 Twenty-three environment variables `config.py` reads were undocumented

Severity: Low **Status:** Closed

Condition. Twenty-three settings — among them the concurrency cap, both rate-limit thresholds, the OpenAI request timeout and retry count, and the data-retention window — had defaults in code but appeared in neither the environment template nor the project documentation, discoverable only by reading the settings module directly.

Remediation. All twenty-three were added to the environment template as commented optional entries, grouped alongside the related settings already documented there, each with a one-line description of what the code actually does with it.

H · Dashboard front end

H1 No ESLint configuration existed; `next lint` was a no-op

Severity: Medium **Status:** Closed

Condition. Both the lint tooling and an npm script to run it were present, but no configuration file existed for it to use — so the script would prompt interactively for one, which hangs or fails outright anywhere non-interactive, including any continuous-integration run.

Remediation. Added a standard configuration extending the framework’s own recommended rules. Running it surfaced two pre-existing lint errors (unescaped apostrophes), which were corrected; the dashboard now lints clean.

H2 A Settings change did not propagate to the rest of the dashboard without a reload

Severity: Medium **Status:** Closed

Condition. Renaming the agent or the company from the Settings page updated the stored value, but the sidebar, the documents view and the transcript view all continued to display the old value until the page was reloaded — at odds with the application’s own stated design of live configuration with no rebuild required.

Remediation. The shared configuration context now exposes a refresh function, called after a successful save of either field, so the change is reflected immediately everywhere the context is consulted.

H3 Four dashboard views were exposed to out-of-order response races

Severity: Medium **Status:** Closed

Condition. The calls, leads and meetings list views, and the call transcript modal, each issued a fetch keyed to the current page, filter or selection with no guard against a slower, earlier request resolving after a faster, later one and overwriting current state with stale data — most easily triggered by a user paging or filtering quickly.

Remediation. All four now apply the same cancellation-flag pattern already used correctly elsewhere in the codebase, so a response belonging to a superseded request is discarded rather than applied.

H4 The analytics page’s “live” feed never actually refreshed

Severity: Low **Status:** Closed

Condition. The page is labelled as a live event feed, but fetched its data once on load and never again without a manual reload, unlike the main dashboard view, which already polled correctly.

Remediation. Brought into line with the dashboard’s existing polling interval and cleanup behaviour.

H5 A settings label was not bound to its control; a failed escalation resolve gave no visible error

Severity: Low **Status:** Closed

Condition. Two independent, minor gaps: a settings-row label sat visually beside its input without a programmatic association, which a screen reader will not reliably connect to the control; and reverting an escalation’s optimistic “resolved” state after a failed save gave no indication to the user of why the card had changed back.

Remediation. Added the missing `id/for` association; wired the existing error-display pattern used elsewhere in the dashboard into the failure path.

H6 Six installed dependencies were never imported anywhere in the dashboard

Severity: Low **Status:** Closed

Condition. Six packages remained in the dependency manifest with no import anywhere in the application, apparently left over from an earlier component-library approach that was replaced by hand-built equivalents.

Remediation. Removed, and the lockfile regenerated to match.

9.5 Matters referred to the client

Two items in this addendum were not treated as ordinary defects to patch, because closing them required a decision about the system’s architecture rather than a correction to its behaviour.

The scope of the public ingress (E1). The narrower alternative to E1’s remediation was removing the `api.yourdomain.com` rule outright. That was not done, because Twilio’s webhook and media-stream connections have no way to reach the backend except directly, and removing the rule would have stopped the application answering calls at all. The client confirmed the narrowing approach — keep the rule, restrict it to exactly the two paths Twilio requires — in preference to the alternative.

The dashboard login, carried forward from VA-B1. Section 8 of this report already identifies the dashboard’s lack of its own login as the most significant open item in the original register: the backend API is now properly authenticated, but the front end in front of it still authenticates every request it forwards with a single shared administrator credential, regardless of who is asking. E1’s narrowing removes the public-internet path to that credential’s target, which was the most severe way the gap could be reached — but the underlying condition Recommendation 1 in section 7 describes is unchanged. Put to the client directly in this session, the decision was to leave it as is for now, rather than introduce a login system, and to record the limitation rather than build against it. It remains the single highest-priority item outstanding across both this addendum and the original register.

9.6 Verification performed

- Full backend automated test suite: 50 passed, no failures, no change in test count (no new tests were added alongside these fixes — consistent with the process observation already on record in section 6).
- Dashboard production build (`next build`): compiles without error, all eleven routes present.
- Every edited Kubernetes and Compose manifest parsed successfully as YAML after editing.
- The new Alembic revision resolves to a single head (0001 → 0002), confirmed with Alembic’s own history and heads commands; no database was altered in the course of verifying this, since none was reachable from the audit environment.

9.7 Outstanding after this addendum

- The dashboard login (section 9.5) remains open, by the client’s own choice on this occasion, and is unchanged from Recommendation 1 in section 7.
- No tests accompany any of the sixteen fixes in this addendum, for the same reason recorded in section 6 against the round-two work: the suite is a scaffold that passes, not a regression net that grows with the codebase.
- Every other recommendation in section 7 that predates this addendum is unaffected by it and remains outstanding on its own terms.